BitNet: Scaling 1-bit Transformers for Large Language Models

Hongyu Wang∗†‡ Shuming Ma∗† Li Dong† Shaohan Huang†

Huaijie Wang§ Lingxiao Ma† Fan Yang† Ruiping Wang‡ Yi Wu§ Furu Wei†⋄

† Microsoft Research ‡ University of Chinese Academy of Sciences § Tsinghua University

Overview

- Model quantization [FAHA23, CCKS23, XLS+23] has emerged as a promising solution, as it can significantly reduce the memory footprint and computational cost of large-scale models while maintaining competitive performance.
- In this work, we introduce BitNet, a scalable and stable 1-bit Transformer architecture designed for large language models.
- We introduce BitLinear as a drop-in replacement of the *nn.Linear* layer (in PyTorch) in order to train 1-bit weights from scratch.
- BitNet achieves competitive performance while substantially reducing memory footprint and energy consumption, compared to state-of-the-art 8-bit quantization methods and FP16 Transformer baselines.
- BitNet exhibits a scaling law akin to full-precision Transformers, suggesting its potential for effective scaling to even larger language models.

Overview

The problem

- It is expensive to host large language models due to the high inference costs and energy consumption.
- As the size of these models grows, the memory bandwidth required for accessing and processing the model parameters becomes a major bottleneck, limiting the overall inference performance.
- When deploying these models on distributed systems or multi-GPU severs, the inter-device communication overhead can significantly impact the inference latency and energy consumption.
- Why are they talking about inference, the paper is about training?

Overview

The problem

- Most existing quantization approaches for large language models are post-training.
- However, post-training quantization techniques result in a more significant loss of accuracy especially when the precision goes lower, because the model is not optimized for the quantized representation during training (empirical results in evaluation).
- Compared to post-training, quantization-aware training typically results in better accuracy, as the model is trained to account for the reduced precision from the beginning.
- The challenge of quantization-aware training mainly lies in optimization, i.e., the model becomes more difficult to converge as the precision goes lower.
- Besides, it is unknown whether quantization-aware training follows the scaling law of language models.

Overview

Contribution summary

- In this work, we focus on binarization (i.e., 1-bit), which is an extreme case of quantization, applied to large language models.
- More specifically large language models that use a unidirectional decoder (e.g., GTP).
- This work is the first to investigate quantization-aware training for 1-bit large language models (different from post-training quantization).
- We propose BitNet, a 1-bit Transformer architecture for large language models, which aims to scale efficiently in terms of both memory and computation.
- BitNet employs
  - low-precision binary weights and quantized activations,
  - while maintaining high precision for the optimizer states and gradients during training.
- The implementation of the BitNet architecture is quite simple, requiring only the replacement of linear projections (e.g., *nn.Linear* in PyTorch) in the Transformer.
- It complements other acceleration methods for large language models, such as PagedAttention [KLZ+23], FlashAttention [DFE+22, Dao23], and speculative decoding [LKM23].

Overview

Contribution summary

- Experimental results demonstrate that BitNet achieves competitive performance in terms of both perplexity and downstream task accuracy.
- BitNet significantly reduces memory footprint and energy consumption compared to the baselines.
- BitNet follows a scaling law similar to that of full-precision Transformers, indicating that it can be effectively scaled to even larger language models.

Proposed solution

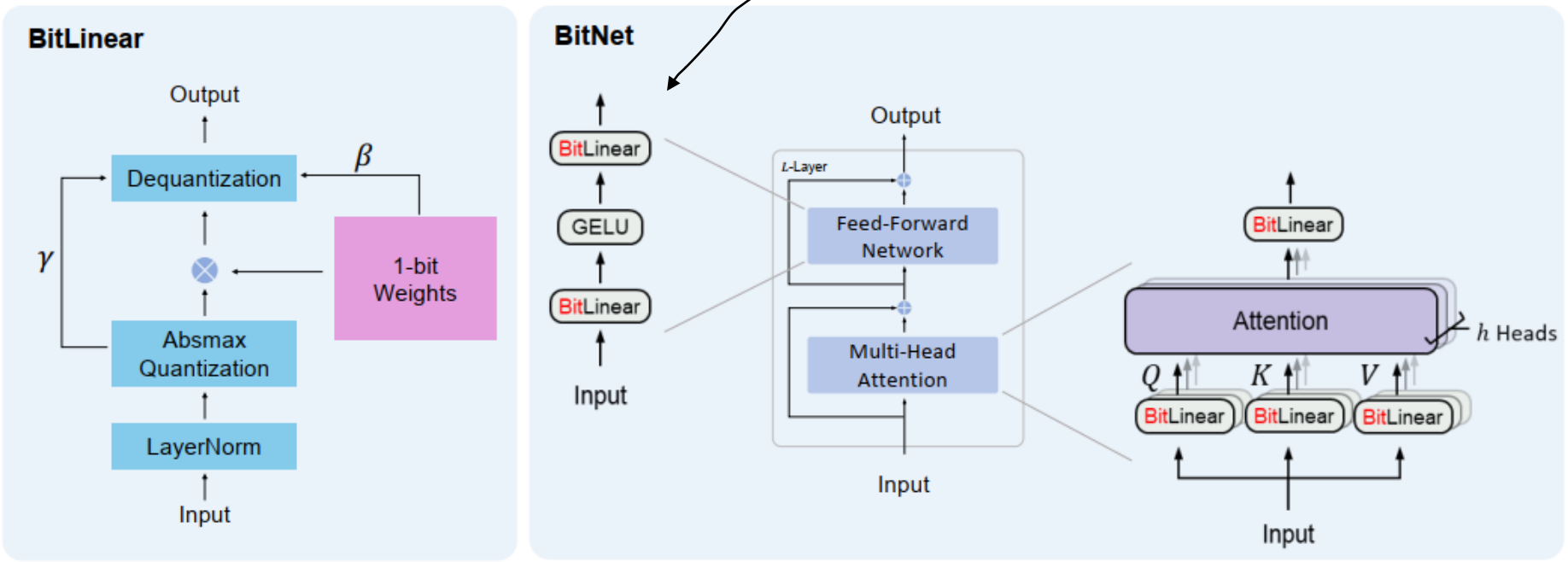
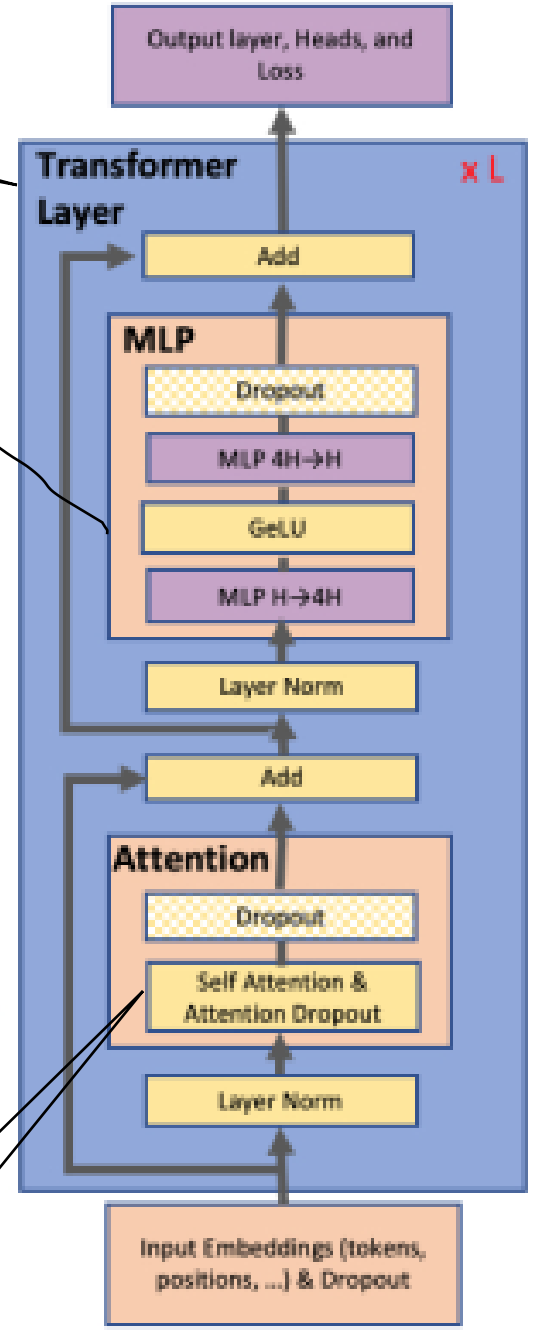


Figure 2: (a) The computation flow of `BitLinear`. (b) The architecture of BitNet, consisting of the stacks of attentions and FFNs, where matrix multiplication is implemented as `BitLinear`.

Vanilla Transformer



Q, K, V

FFN - Feed-Forward Network

⊗ is the tensor product operator

Proposed solution

- BitNet uses the same layout as Transformers, stacking blocks of self-attention and feed-forward networks.
- Compared with vanilla Transformer, BitNet uses BitLinear instead of conventional matrix multiplication, which employs binarized (i.e., 1-bit) model weights (see the equations below).
- We leave the other components high-precision, e.g., 8-bit in our experiments.
- Moreover, the quantization is performed per tensor during training while per token during inference for both stability and efficiency. (More explanation is required.)
- Design motivation:
  - First, the residual connections and the layer normalization contribute negligible computation costs to large language models.
  - Second, the computation cost of QKV transformation is much smaller than the parametric projection (i.e., dimensionality reduction using matrix multiplication) as the model grows larger.
  - Third, we preserve the precision for the input/output embedding because the language models have to use high-precision probabilities to perform sampling.

$$y = \widetilde{W}\widetilde{x} = \widetilde{W}\ \mathrm{Quant}(\mathrm{LN}(x)) \times \frac{\beta\gamma}{Q_b}$$

$$\mathrm{LN}(x) = \frac{x - E(x)}{\sqrt{\mathrm{Var}(x) + \epsilon}}, \quad \beta = \frac{1}{nm}||W||_1$$

Proposed solution

How BitLinear works?

- We first binarize the weights to either +1 or -1 with the signum function
- We centralize the weights to be zero-mean (standard normal distribution) before binarization to increase the capacity within a limited numerical range [LOP+22].
- The binarization of a weight W_{ij} in a weight matrix $W \in R^{n \times m}$ can be formulated as shown.

- A scaling factor β is used after binarization to reduce the $L2$ error (MSE - mean squared error) between the real-valued and the binarized weights.

We want to compute, which replaces conventional matrix multiplication

$$y = \widetilde{W}\widetilde{x}$$

Original weight matrix is W

Binarized weights $\widetilde{W} = \text{Sign}(W - \alpha),$

$$\text{Sign}(W_{ij}) = \begin{cases} +1, & \text{if } W_{ij} > 0, \\ -1, & \text{if } W_{ij} \leq 0, \end{cases}$$

$$\alpha = \frac{1}{nm} \sum_{ij} W_{ij}$$

Proposed solution

We want to compute, which replaces conventional matrix multiplication

$$y = \widetilde{W}\widetilde{x}$$

Original input matrix is *x*

Quantized activation $\widetilde{x} = \text{Quant}(x)$

How BitLinear works?

- We quantize the activations to *b*-bit precision (e.g., FP8).
- We use absmax quantization, which scales activations into the range [$-Q_b$, Q_b]. Here, ($Q_b = 2^b-1$).
- This is doen by multiplying with Q_b and dividing by γ, the absolute maximum of the input matrix *x*.
- ϵ is a small floating-point number that prevents overflow when performing the clipping.

$$\widetilde{x} = \text{Quant}(x) = \text{Clip}(x \times \frac{Q_b}{\gamma}, -Q_b + \epsilon, Q_b - \epsilon),$$

$$\text{Clip}(x, a, b) = \max(a, \min(b, x)), \quad \gamma = ||x||_\infty,$$

Infinity norm, $\|x\|_\infty = \max_i(|x_i|)$

- For the activations before the non-linear functions (e.g., GeLU), we scale them into the range [0, Q_b] by subtracting the minimum of the inputs η so that all values are non-negative.

$$\widetilde{x} = \text{Quant}(x) = \text{Clip}((x - \eta) \times \frac{Q_b}{\gamma}, \epsilon, Q_b - \epsilon), \quad \eta = \min_{ij} x_{ij}.$$

Proposed solution

We want to compute, which replaces conventional matrix multiplication

$$y = \widetilde{W}\widetilde{x}$$

How BitLinear works?

- The variance *Var*(*y*) of the output *y* is estimated as shown.

$$\begin{aligned}\mathrm{Var}(y) &= n\mathrm{Var}(\widetilde{w}\widetilde{x}) \\ &= nE[\widetilde{w}^2]E[\widetilde{x}^2] \\ &= n\beta^2 E[\widetilde{x}^2] \approx E[\widetilde{x}^2]\end{aligned}$$

In probability and statistics, *E*[x] represents mean or expected value of a random variable x

$$\beta = \frac{1}{nm}||W||_1$$

L1 norm, $\|W\|_1 = \sum_i^n |W_i|$

- To preserve the variance after quantization, we introduce a LayerNorm [BKH16] function before the activation quantization. $\mathrm{Quant}(\mathrm{LN}(x))$
- In the context of Transformers, LayerNorm has the exact implementation as SubLN [WMH+22].

The variance of the output *y* is estimated as the following which has the same magnitude as the full-precision counterpart:

$$\mathrm{Var}(y) \approx E[\mathrm{LN}(\widetilde{x})^2] = 1$$

We want to compute, which replaces conventional matrix multiplication

$$y = \widetilde{W}\widetilde{x}$$

Proposed solution

How BitLinear works?

- In the context of Transformers, LayerNorm has the exact implementation as SubLN [WMH+22].
- With SubLN and the quantization methods described earlier, we have BitLinear which is formulated as the following.
- A scaling factor β is used after binarization to reduce the $L2$ error (MSE - mean squared error) between the real-valued and the binarized weights.

$$y = \widetilde{W}\widetilde{x} = \widetilde{W}\ \mathrm{Quant}(\mathrm{LN}(x)) \times \frac{\beta\gamma}{Q_b}$$

$$\mathrm{LN}(x) = \frac{x - E(x)}{\sqrt{\mathrm{Var}(x) + \epsilon}}, \quad \beta = \frac{1}{nm}||W||_1$$

L1 norm, $\|W\|_1 = \sum_i^n |W_i|$

- Figure 2 provides an illustration of the computation flow of BitLinear.
- After the SubLN operation (LayerNorm), the activations are quantized with the absmax function.
- The matrix multiplication ⊗ is performed between the 1-bit weights and the quantized activations.
- The output activations are rescaled with $\{\beta, \gamma\}$ to dequantize them to the original precision.

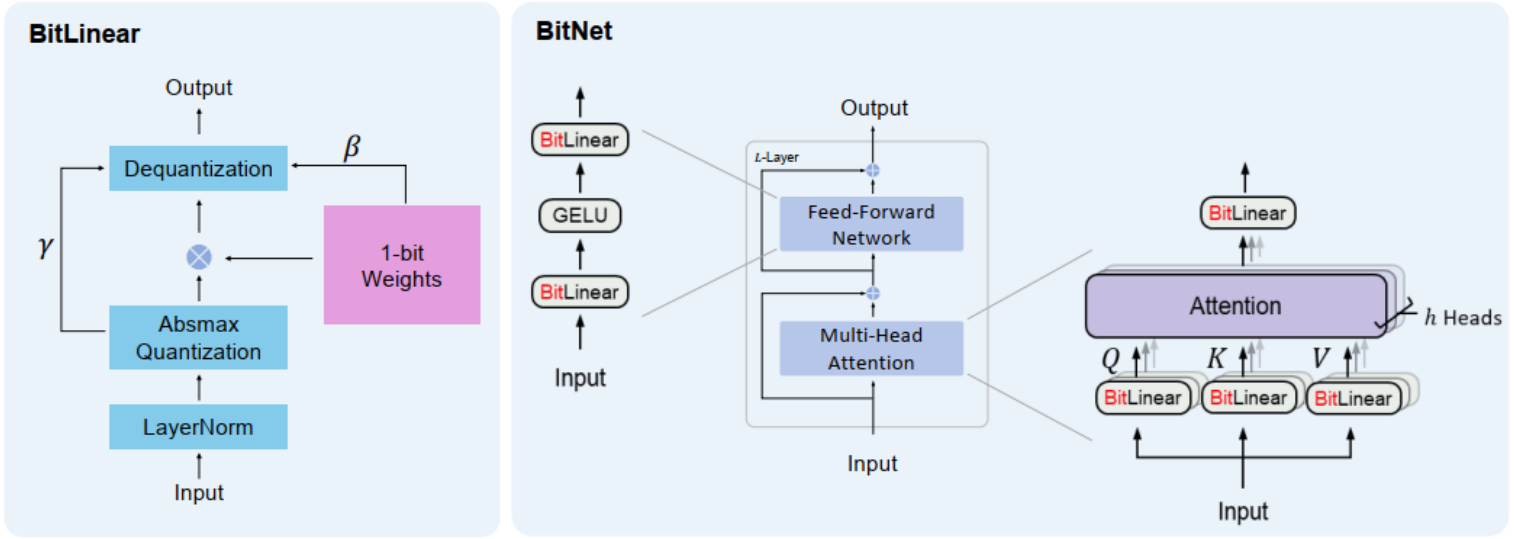


Figure 2: (a) The computation flow of `BitLinear`. (b) The architecture of BitNet, consisting of the stacks of attentions and FFNs, where matrix multiplication is implemented as `BitLinear`.

Proposed solution

Model parallelism with group quantization and normalization

- A prerequisite for the existing model parallelism approaches is that the tensors are independent along the partition dimension (e.g., tensor parallelism in Megatron-LM). (How would you make BitNet work for data and pipeline parallelism?)
- However, all of the parameters α, β, γ, and η are calculated from the whole tensors, breaking the independent prerequisite. A naïve approach would incur significant synchronization overhead.
- We propose a solution, called Group Quantization, that makes the model parallelism more efficient when using BitNet.

Proposed solution

Model parallelism with group quantization and normalization

- We divide the weights and activations into groups and then independently estimate each group's parameters, so the parameters can be calculated locally without requiring additional communication.
- We divide a weight matrix $W \in R^{n \times m}$ into G groups along the partition dimension.
- Each group has a size of $n/G \times m$. We then estimate the parameters for each group independently. Here, $W^{(g)}$ denotes the g-th group of the weight matrix.

$$\alpha_g = \frac{G}{nm} \sum_{ij} W_{ij}^{(g)}, \quad \beta_g = \frac{G}{nm} ||W^{(g)}||_1,$$

Proposed solution

Model parallelism with group quantization and normalization

- Similarly, for the activations, we can divide the input matrix $x \in R^{n \times m}$ into G groups and calculate the parameters for each group.

$$\gamma_g = ||x^{(g)}||_\infty, \quad \eta_g = \min_{ij} x_{ij}^{(g)}$$

- For LN, we can apply the group normalization technique described in [WH20] to compute the mean and variance for each group independently.

$$\mathrm{LN}(x^{(g)}) = \frac{x^{(g)} - E(x^{(g)})}{\sqrt{\mathrm{Var}(x^{(g)}) + \epsilon}}$$

Proposed solution

Model training and inference

- To train our 1-bit model, we employ the straight-through estimator (STE)[BLC13] to approximate the gradient during backpropagation.
- This method bypasses the nondifferentiable functions, such as the Sign (Eq. 2) and Clip (Eq. 5) functions, during the backward pass.
- STE allows gradients to flow through the network without being affected by these non-differentiable functions, making it possible to train our quantized model.
- While the weights and the activations are quantized to low precision, the gradients and the optimizer states are stored in high precision to ensure training stability and accuracy.
- We maintain a latent weight in a high-precision format for the learnable parameters to accumulate the parameter updates [LSL+21].
- The latent weights are binarized on the fly during the forward pass and never used for the inference process.

Evaluation

Computational efficiency

- We estimate the computational efficiency of BitNet in terms of both arithmetic operations energy and memory footprint. We mainly focus on the calculation for the matrix multiplication, since it contributes the most to the cost of large language models.
- In vanilla Transformers, for matrix multiplication with dimensions $m \times n$ and $n \times p$, the energy consumption can be calculated as follows:

$$E_{add} = m \times (n-1) \times p \times \hat{E}_{add}$$
$$E_{mul} = m \times n \times p \times \hat{E}_{mul}$$

For BitNet, the energy consumption of the matrix multiplication is dominated by the addition operations, as the weights are 1-bit. The multiplication operations are only applied to scale the output with the scalars β and $\frac{\gamma}{Q_b}$, so the energy consumption for multiplication can be computed as:

$$E_{mul} = (m \times p + m \times n) \times \hat{E}_{mul}$$

Arithmetic operations energy. According to the energy model in [Hor14, ZZL22], the energy consumption for different arithmetic operations can be estimated as follows:

Bits	ADD Energy $\hat{E}_{add}$ (pJ)		MUL Energy $\hat{E}_{mul}$ (pJ)	
	45nm	7nm	45nm	7nm
FP32	0.9	0.38	3.7	1.31
FP16	0.4	0.16	1.1	0.34
INT8	0.03	0.007	0.2	0.07

Table 2: ADD and MUL energy consumption [Hor14, ZZL22] for different bit representations at 45nm and 7nm process nodes.

Evaluation

Computational efficiency

- The energy savings of W1A8 BitNet compared to a full-precision (W32A32) and half-precision (W16A16) Transformer are shown in Table 1.

Models	Size	WBits	7nm Energy (J)		45nm Energy (J)	
			MUL	ADD	MUL	ADD
Transformer	6.7B	32	4.41	1.28	12.46	3.03
		16	1.14	0.54	3.70	1.35
BitNet		1	**0.02**	**0.04**	**0.08**	**0.13**
Transformer	13B	32	8.58	2.49	24.23	5.89
		16	2.23	1.05	7.20	2.62
BitNet		1	**0.04**	**0.06**	**0.12**	**0.24**
Transformer	30B	32	20.09	5.83	56.73	13.80
		16	5.21	2.45	16.87	6.13
BitNet		1	**0.06**	**0.14**	**0.20**	**0.53**

Table 1: Energy consumption of BitNet and Transformer varying different model size. Results are reported with 512 as input length.

Evaluation

Inference-optimal scaling law

- It has been shown that in LLMs, the loss scales as the power law with the amount of computation used for training [KMH+20].
- This allows us to determine the optimal allocation of a computation budget as well as predict the performance of large language models from smaller models.
- We fix the number of training tokens and vary the model sizes. Figure 3 shows that the loss scaling of BitNet is similar to the FP16 Transformer, which follows a power-law.

- While the power-law measures the trend of the scaling of BitNet, it does not properly model the relationship between the loss and the actual compute.
- Previous work [KMH+20, HKK+20, HBM+22] estimates the compute by calculating the FLOPs.
- However, it does not apply to 1-bit models whose cost is dominated by integer computation. Moreover, it mainly measures the training computation rather than the inference.

Evaluation

Inference-optimal scaling law

- To have a better understanding of the scaling efficiency of neural language models, we introduce Inference-Optimal Scaling Law. It predicts the loss against the energy consumption. (See the slide about computational efficiency.)
- Figure 3 shows the scaling curve against the inference energy cost at 7nm process nodes (see Table 2).

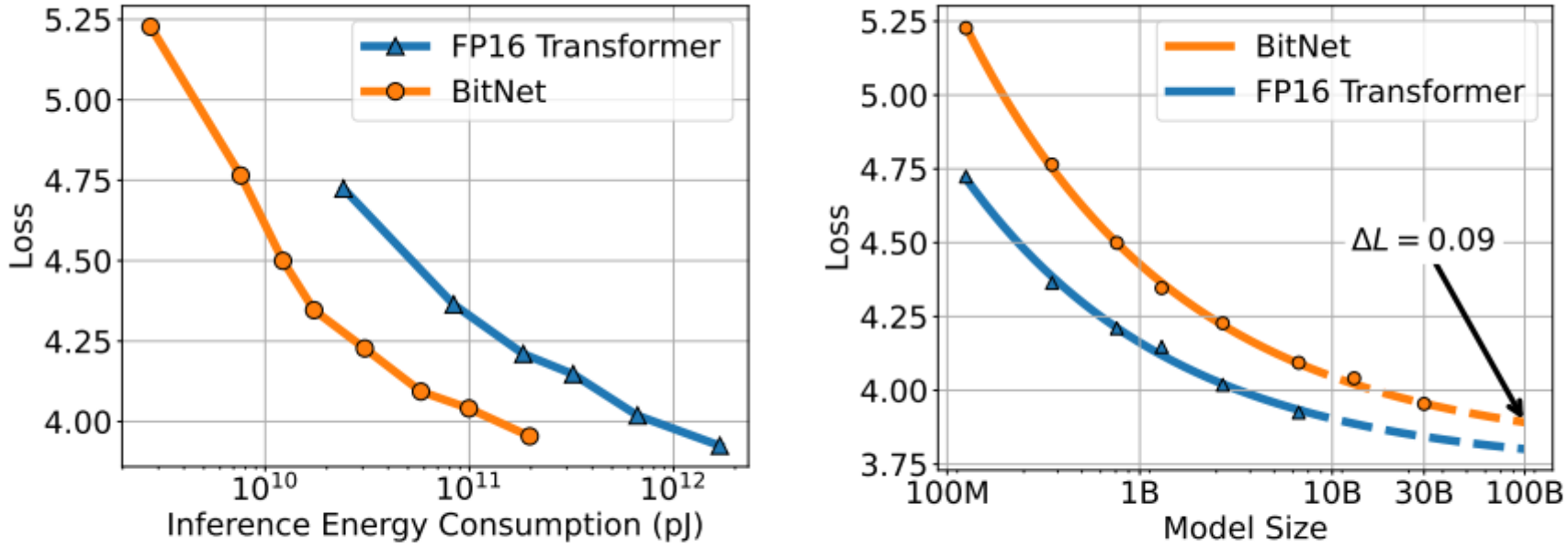


Figure 3: Scaling curves of BitNet and FP16 Transformers.

Evaluation

Results on downstream tasks

- To evaluate the capabilities with the interpretable metrics, we test both the 0-shot and 4-shot results on four downstream tasks, including Hellaswag [ZHB+19], Winogrande [SBBC20], Winograd [LDM12], and Storycloze [MCH+16].
- Figure 4 reports the average results of BitNet and FP16 Transformer with various scales.
- Similar to the loss scaling curve, the performance on the downstream tasks can scale as the computation budget grows.
- Besides, the scaling efficiency of capabilities is much higher than the FP16 Transformer baseline, in terms of both zero-shot and few-shot performance.

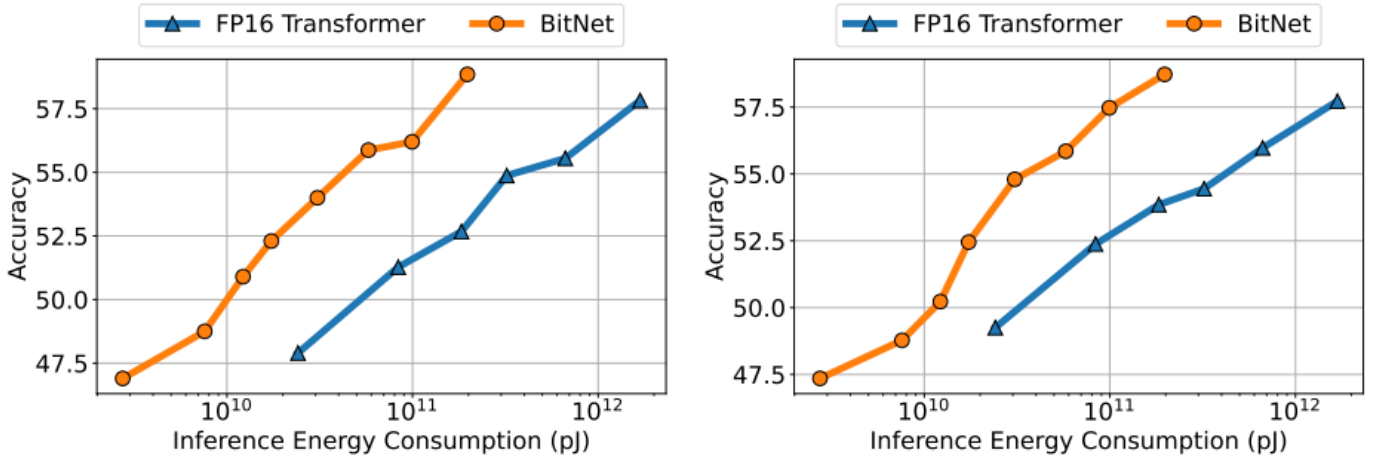


Figure 4: Zero-shot (Left) and few-shot (Right) performance of BitNet and FP16 Transformer against the inference cost.

Evaluation

Stability test

- The major challenge for training low-bit Transformers is the stability in optimization. Therefore, we perform stability tests for both BitNet and the FP16 baseline by training a series of models with varying peak learning rates (lr).
- It shows that BitNet can converge with a large learning rate while FP16 Transformer can not, demonstrating better training stability of BitNet. This advantage in optimization enables the training with larger learning rates. Here, PPL is short for perplexity, lower is better. The perplexity metric indicates inference performance of an NLP model.

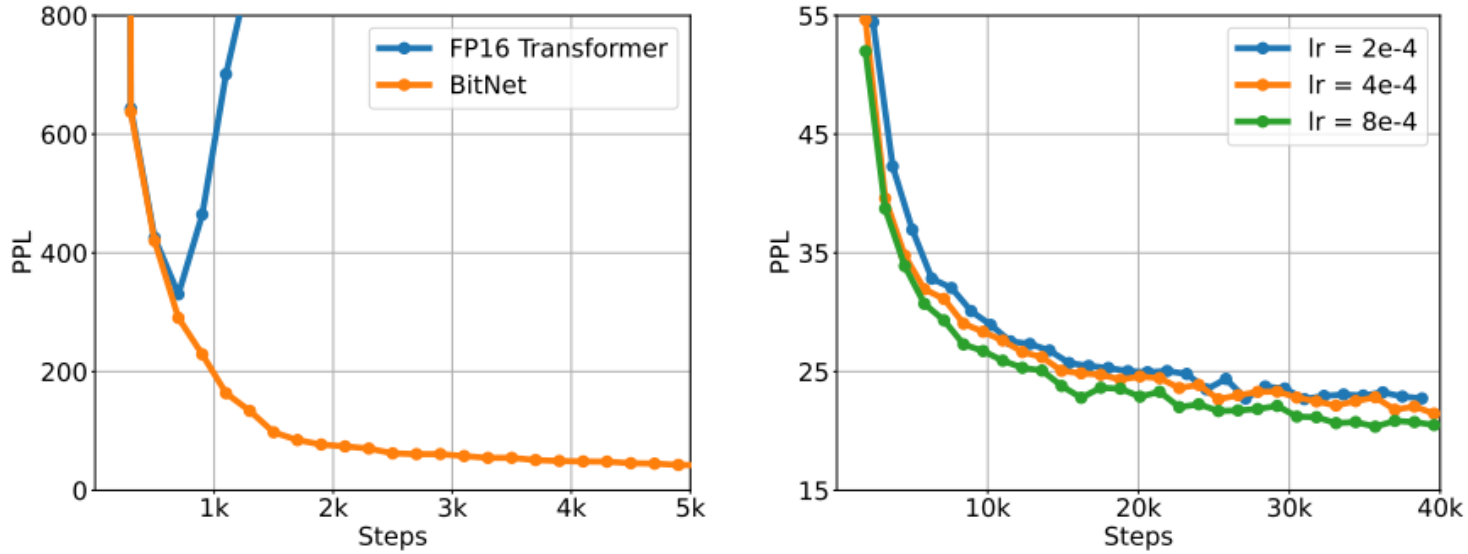


Figure 5: BitNet is more stable than FP16 Transformer with a same learning rate (Left). The training stability enables BitNet a larger learning rate, resulting in better convergence (Right).

Evaluation

Comparison with post-training quantization

- We compare BitNet with state-of-the art quantization methods, including Absmax [DLBZ22], SmoothQuant [XLS+23], GPTQ [FAHA23], and QuIP [CCKS23]. These methods are post-training quantization over an FP16 Transformer model.
- Absmax and SmoothQuant quantize both the weights and the activations, while GPTQ and QuIP only reduce the precision of weights.
- For the weight-only quantization (i.e., GPTQ and QuIP), we experiment with W4A16 and W2A16 (W4A16 - weight is 4-bit, activation is FP16). For weight-and-activation quantization (i.e., Absmax and SmoothQuant), we use them to quantize the FP16 Transformers to W8A8, W4A4, and W1A8.
- Our implementation of BitNet is binary weight 8-bit activation (W1A8).

Evaluation

Comparison with post-training quantization

- All models have the model sizes of 6.7B.
- The methods are evaluated across several weight bit levels, spanning from 16 down to 1.
- Besides the zero-shot accuracy on the downstream tasks, the evaluation metrics include language model perplexity (PPL) on the validation set.

Evaluation

Comparison with post-training quantization

- Table 3 presents a detailed comparative analysis of the zero-shot performance of our proposed method. (PPL - perplexity (lower is better), Wbits - #bits used to represent weight value.)

WBits	Methods	PTQ	PPL↓	WG↑	WGe↑	HS↑	SC↑	Avg↑
-	Random	✗	-	50.0	50.0	25.0	50.0	43.8
16	Transformer	✗	15.19	66.7	54.3	42.9	67.4	57.8
8	Absmax	✓	21.43	60.4	52.0	38.3	62.7	53.4
	SmoothQuant	✓	15.67	65.3	53.1	40.9	67.6	56.7
4	GPTQ	✓	16.05	57.2	51.2	39.9	63.4	52.9
	Absmax	✓	4.8e4	55.8	50.9	25.0	53.1	46.2
	SmoothQuant	✓	1.6e6	53.7	48.3	24.8	53.6	45.1
2	GPTQ	✓	1032	51.6	50.1	25.8	53.4	45.2
	QuIP	✓	70.43	56.1	51.2	30.3	58.4	49.0
1	Absmax	✓	3.5e23	49.8	50.0	24.8	53.6	44.6
	SmoothQuant	✓	3.3e21	50.5	49.5	24.6	53.1	44.4
1	**BitNet**	✗	17.07	66.3	51.4	38.9	66.9	55.9

Table 3: Zero-shot results for BitNet and the baselines (PTQ: Post-training quantization, WGe: Winogrande, WG: Winograd, SC: Storycloze, and HS: Hellaswag dataset).

Evaluation

Comparison with post-training quantization

- Figure 6 summarizes both the zero-shot accuracy and few-shot accuracy of our method and the baselines while scaling up the model size from 1.3B to 6.7B.

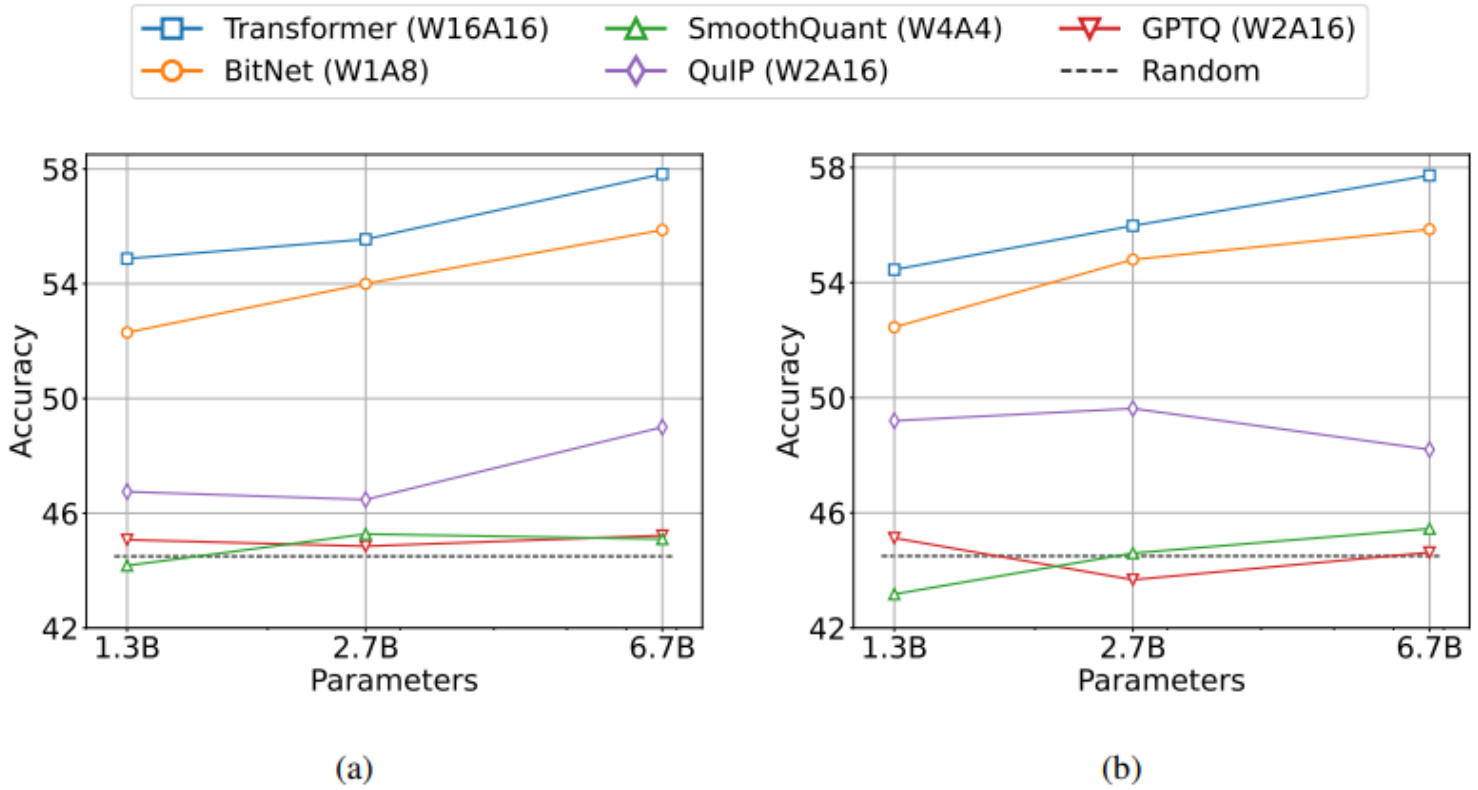


Figure 6: Zero-shot (Left) and few-shot (Right) results for BitNet and the post-training quantization baselines on downstream tasks.

Evaluation

Comparison with post-training quantization

Observations:

- The zero-shot scores of BitNet are comparable with the 8-bit models, while the inference cost is much lower.
- For the 4-bit models, the weight-only quantization methods outperform the weight-and-activation quantizers, mainly because the activation is more difficult to quantize.
- BitNet, as a 1-bit model, significantly achieves better results than both the weight-and-activation quantization methods and the weight-only methods.
- As for the lower-bit models, BitNet has consistently superior scores over all baselines.
- This proves the advantages of the quantization-aware training approaches over the post-training quantization methods.

Evaluation

Ablation studies

- Comparison with several alternative approaches.
- BitNet implements absmax to quantize the activation and uses SubLN for training stability.
- One quantization alternative is the elastic function [LOP+22], which dynamically adjusts the scales with learnable parameters.
- Pre-LN is the default architecture for GPT pertaining, while BMT has proven to improve the stability of binarized models. The results show that BitNet outperforms both.

Methods	PPL↓	HS↑	WGe↑	WG↑	SC↑	Avg↑
Zero-Shot Learning						
BitNet	**20.34**	33.2	52.1	60.7	63.2	**52.3**
Elastic + Pre-LN	24.05	29.6	52.9	56.8	61.3	50.2
Absmax + Pre-LN	22.11	31.6	50.0	61.8	61.6	51.3
Absmax + BMT	22.98	31.2	52.1	60.4	62.7	51.6
Few-Shot Learning						
BitNet	**20.34**	33.5	50.4	62.1	63.8	**52.5**
Elastic + Pre-LN	24.05	29.9	51.7	57.5	61.1	50.1
Absmax + Pre-LN	22.11	31.4	51.9	63.9	61.6	52.2
Absmax + BMT	22.98	31.3	51.5	57.5	62.6	50.7

Table 4: Ablation of BitNet (WGe: Winogrande, WG: Winograd, SC: Storycloze, and HS: Hellaswag dataset). Elastic is an activation quantization method from [LOP+22], while BMT is the architecture from [ZGC+23] to stabilize the training of low-bit models.